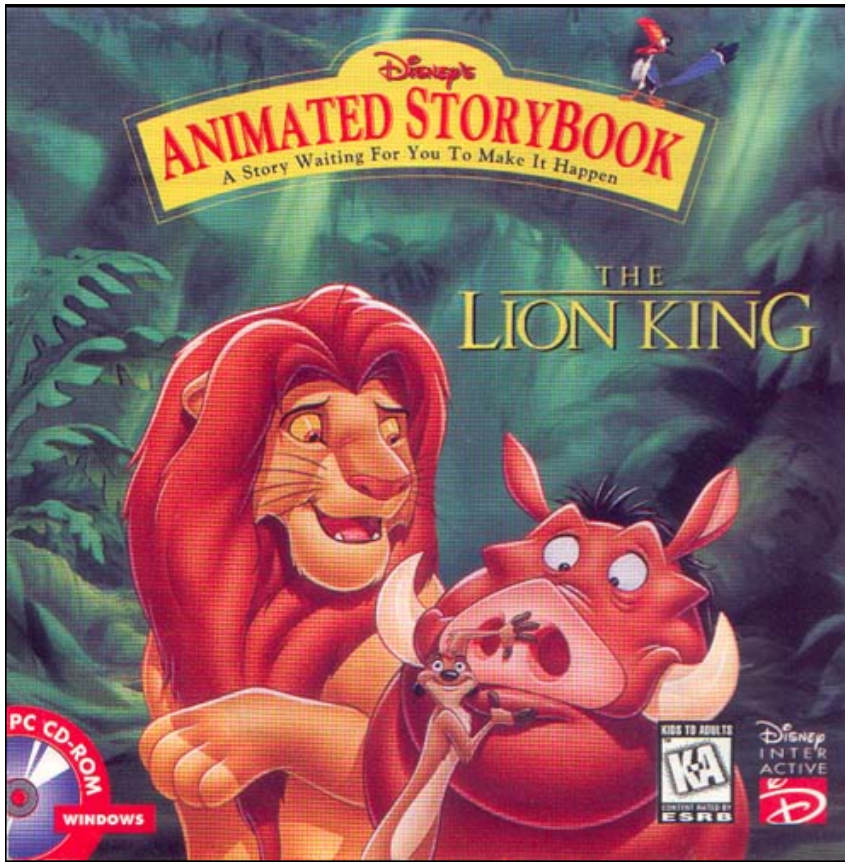
第六章:软件测试及代码质量保障

第六章：软件测试及代码质量保障

- 1. 软件测试的定义和分类
- 2. 测试用例的定义和其设计方法
- 3. 白盒测试（定义、控制流程图、逻辑覆盖方法、测试用例设计）
- 4. 黑盒测试（定义、等价类划分、边界值分析、场景法）
- 5. 压力测试，性能测试和代码覆盖率测试
- 6. 代码质量保证

软件缺陷的典型案例分析



狮子王动画故事书





波音737-800MAX

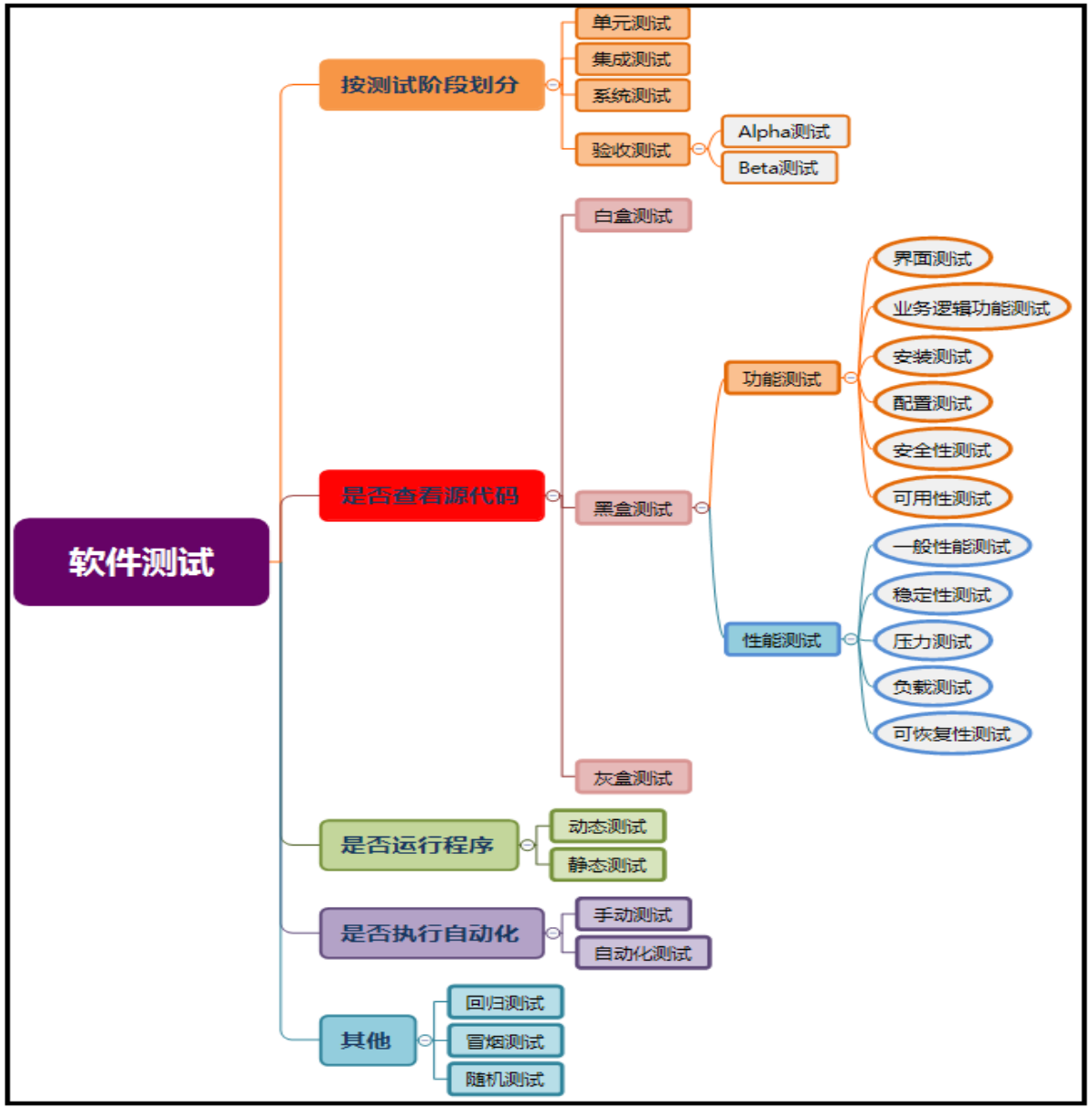
软件测试的定义(IEEE)

使用人工或自动的手段来运行或测定被测系统，或静态检查被测系统的过程，其目的在于校验被测系统是否满足需求，并找出实际系统输出和预期结果之间的差异。软件测试是以需求为中心，并非以缺陷为中心！

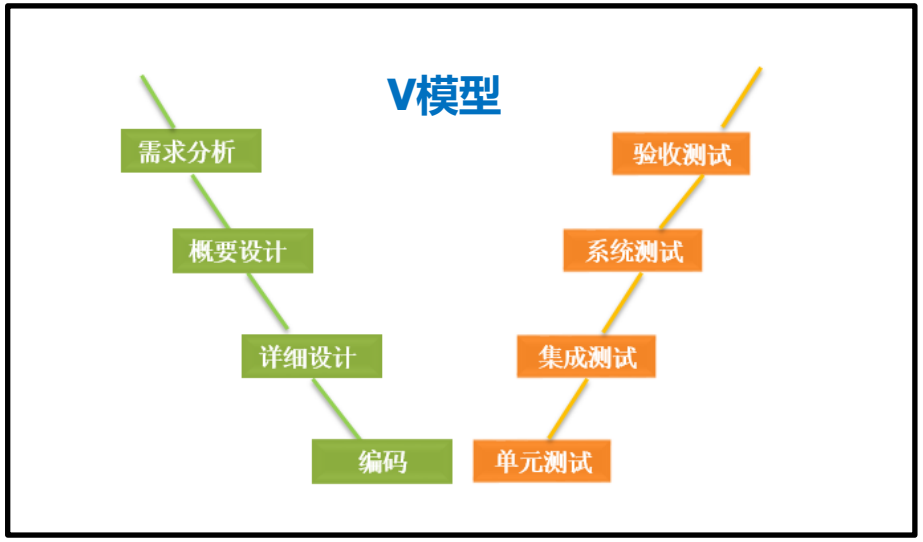


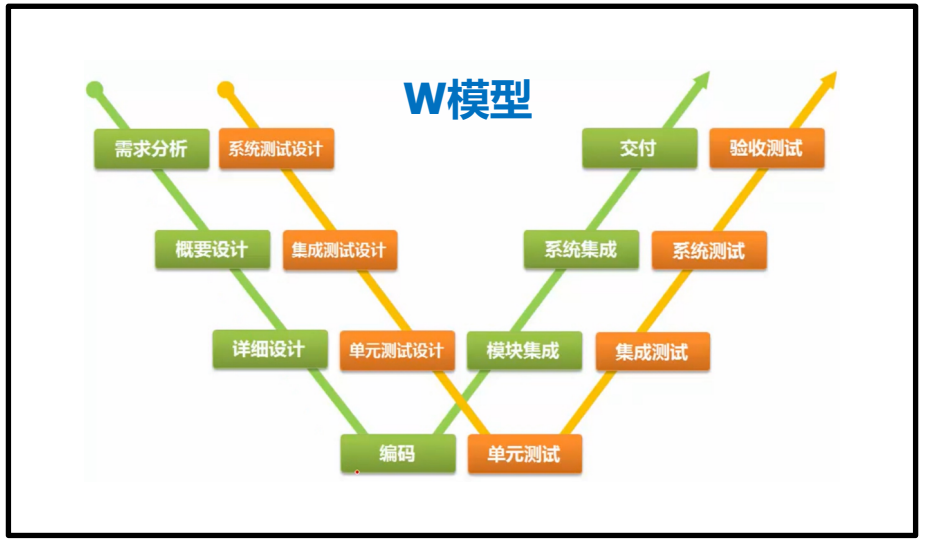
软件测试的分类

软件生命周期中将会执行不同类型或级别的软件测试，组合应用这些不同类型的测试才能确保软件系统功能和行为符合预期要求。



软件测试的分类





- 单元测试对应编码阶段，其测试对象是单个模块或组件；
- 集成测试对应详细设计，其测试对象是一组模块或组件；
- 系统测试和验收测试分别对应概要设计和需求阶段，测试对象是整个系统。

测试用例：定义

测试用例(Test Case)是指对一项特定的软件产品进行测试任务的描述，体现测试方案、方法、技术和策略。

简单地认为，测试用例是为某个特殊目标而编制的一组测试输入、执行条件以及预期结果，用于核实是否满足某个特定软件需求。

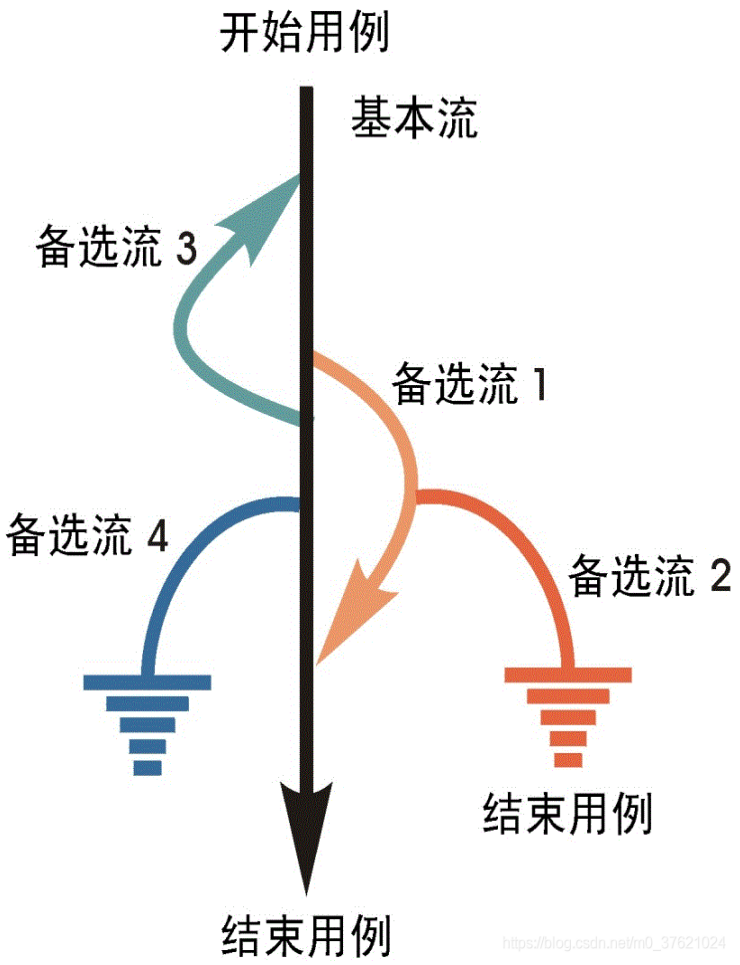
测试用例是执行的最小测试实体。

测试用例：设计方法

测试用例（Test Case）是将软件测试的行为活动做一个科学化的组织归纳，目的是能够将软件测试的行为转化成可管理的模式；同时测试用例也是将测试具体量化的方法之一，不同类别的软件，测试用例是不同的。

测试用例的设计方法主要有黑盒测试法和白盒测试法。

黑盒测试也称功能测试，黑盒测试着眼于程序外部结构，不考虑内部逻辑结构，主要针对软件界面和软件功能进行测试。
白盒测试又称结构测试、透明盒测试、逻辑驱动测试或基于代码的测试。白盒法全面了解程序内部逻辑结构、对所有逻辑路径进行测试。

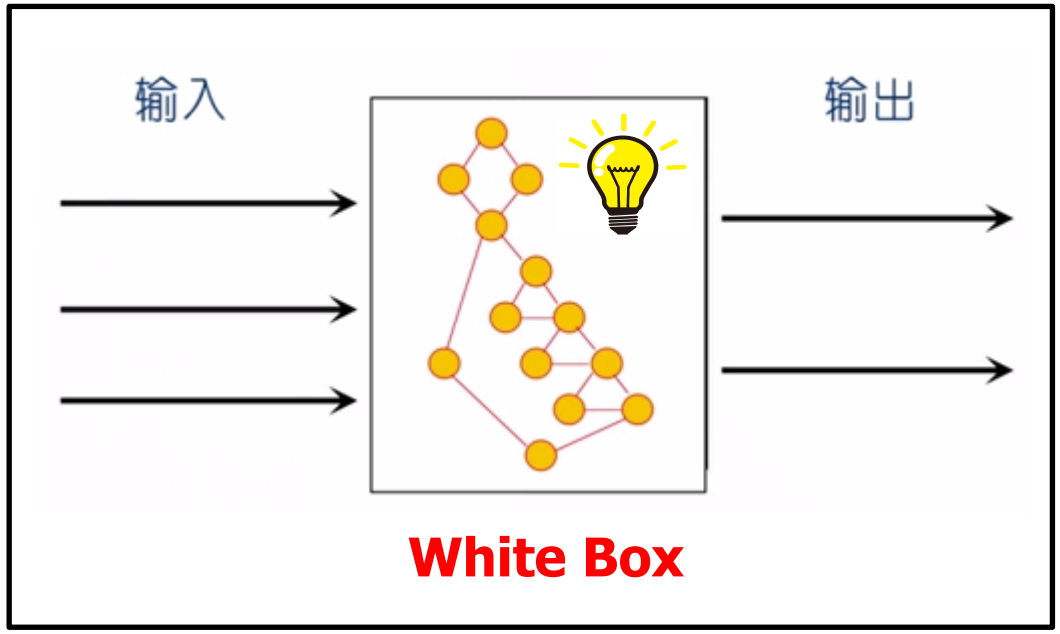


测试用例：设计原则

- 测试用例设计一般遵循以下原则：
  - **正确性。**输入用户实际数据以验证系统是否满足需求规格说明书的要求;测试用例中的测试点应首先保证要至少覆盖需求规格说明书中的各项功能，并且正常。
  - **全面性。**覆盖所有的需求功能项;设计的用例除对测试点本身的测试外，还需考虑用户实际使用的情况、与其他部分关联使用的情况、非正常情况(不合理、非法、越界以及极限输入数据)操作和环境设置等。
  - **连贯性。**用例组织有条理、主次分明，尤其体现在业务测试用例上;用例执行粒度尽量保持每个用例都有测点，不能同时覆盖很多功能点，否则执行起来牵连太大，所以每个用例间保持连贯性很重要。
  - **可判定性。**测试执行结果的正确性是可判定的，每一个测试用例都有相应的期望结果。
  - **可操作性。**测试用例中要写清楚测试的操作步骤，以及与不同的操作步骤相对应的测试结果。

白盒测试：定义

也称作结构测试或逻辑驱动测试，它是基于程序的源代码，已知产品的内部工作过程，主要是对程序内部结构展开测试，关注程序实现细节，检验程序中的每条通路是否都有按照预定要求正确工作。



优势：

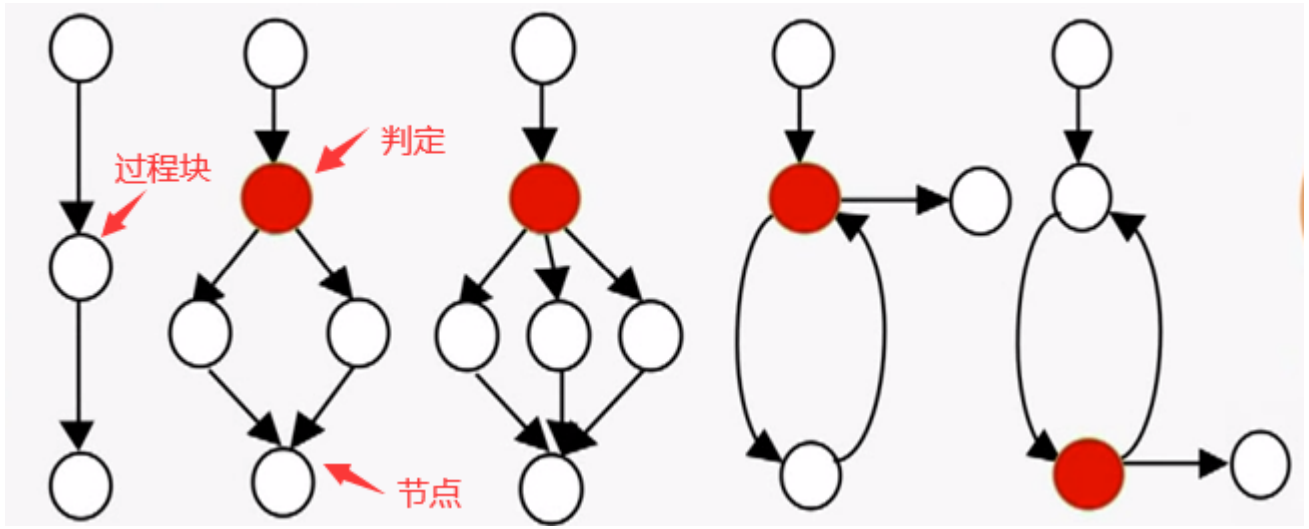
- 针对性强，可快速定位Bug
- 函数级别，Bug修复成本低
- 有助于了解测试的覆盖程度
- 有助于优化代码，预防缺陷

劣势：

- 对测试人员要求高
- 成本高

白盒测试：控制流程图

一种表示程序控制结构的图形工具，其基本元素是节点、**判定节点**、过程块。



缺陷风险：线性结构 < 两分支的条件判定 < 多分支的条件判定 < 循环结构

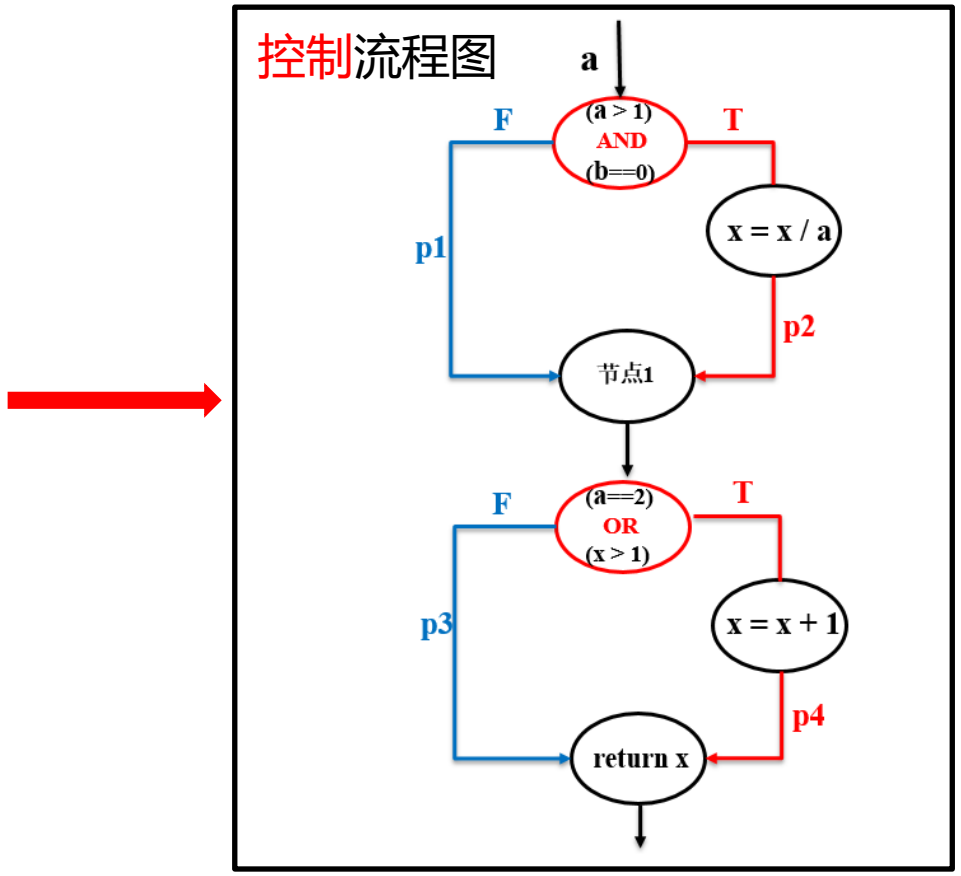
白盒测试：建立被测对象模型

源代码

```
int Func1 (int a, int b, int x)
{
    if ((a > 1) && (b == 0))
        x = x / a;

    if ((a == 2) || (x > 1))
        x = x + 1;

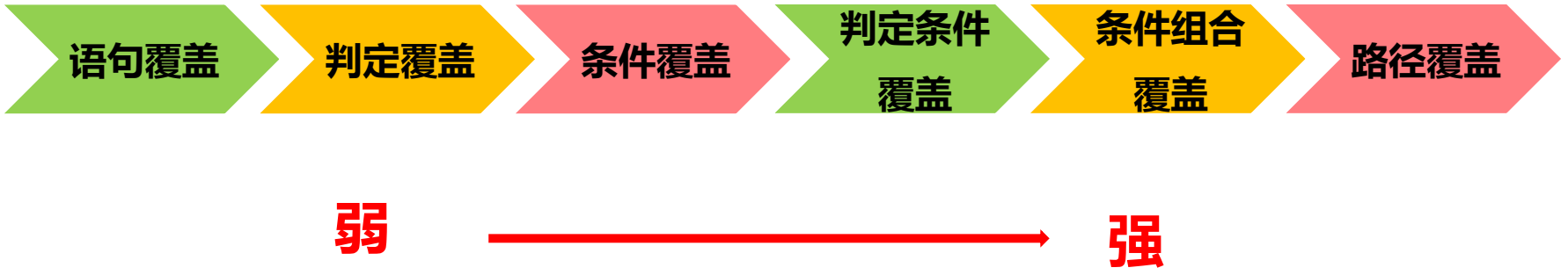
    return x;
}
```



白盒测试：逻辑覆盖方法

对判定的测试

- 关注点：判定表达式本身的复杂度
- 原理：考察源代码中的判定表达式，通过对程序逻辑结构的遍历，来实现测试对程序的覆盖。
- 常见覆盖指标：



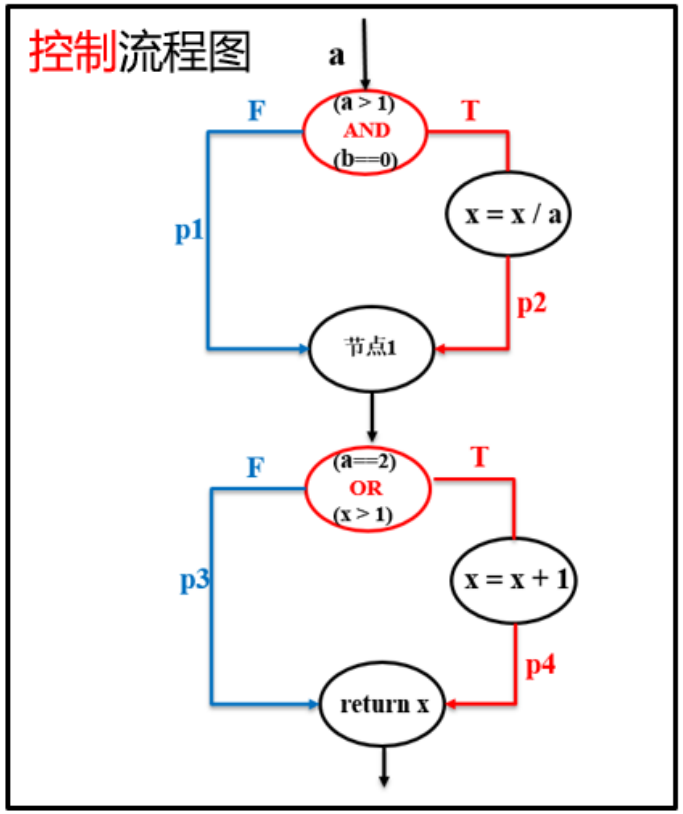
白盒测试：一个例子

- **4个基本逻辑判定条件**

T1	a > 1
T2	b==0
T3	a==2
T4	X>1

- **4条执行路径**

L13	p1->p3
L14	p1->p4
L23	p2->p3
L24	p2->p4



白盒测试：一个例子

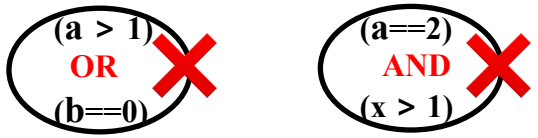
（一）语句覆盖

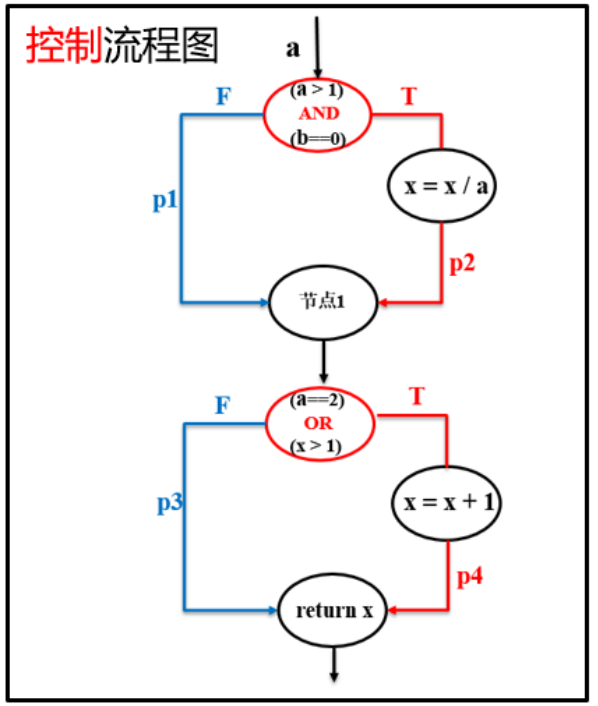
至少执行程序中所有语句一次。语句覆盖是最弱的逻辑覆盖准则。

设计测试用例，实现语句覆盖：

测试用例ID	输入			预期输出	覆盖路径
	a	b	x	x	
TC1	2	0	4	3	L24

测试漏洞：





白盒测试：一个例子

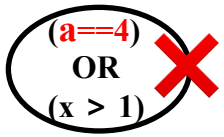
（二）判定覆盖

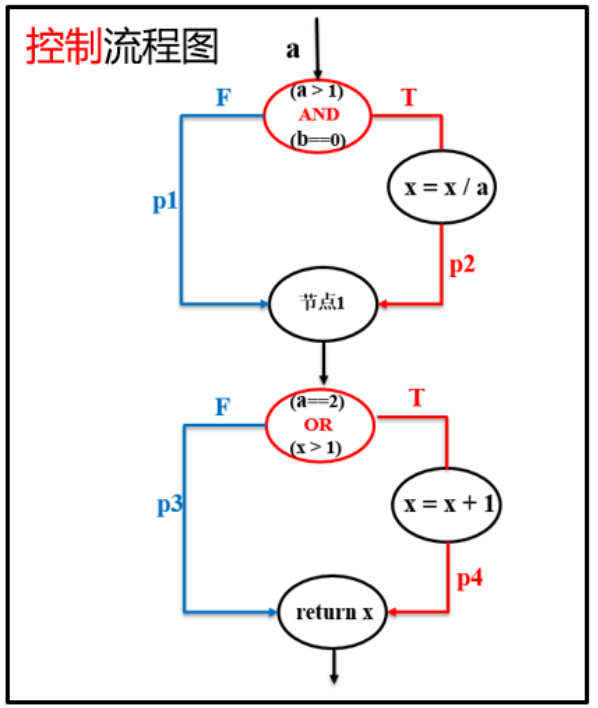
也称为分支覆盖，至少执行程序中每个分支一次，保证程序中每个判定节点取得每种可能的结果至少一次。

设计测试用例，实现判定覆盖：

测试用例ID	输入			预期输出	覆盖路径
	a	b	x	x	
TC1	1	1	2	3	L14
TC2	3	0	3	1	L23

测试漏洞：





白盒测试：互动小问题

（三）判定覆盖

满足判定覆盖是否满足语句覆盖呢？

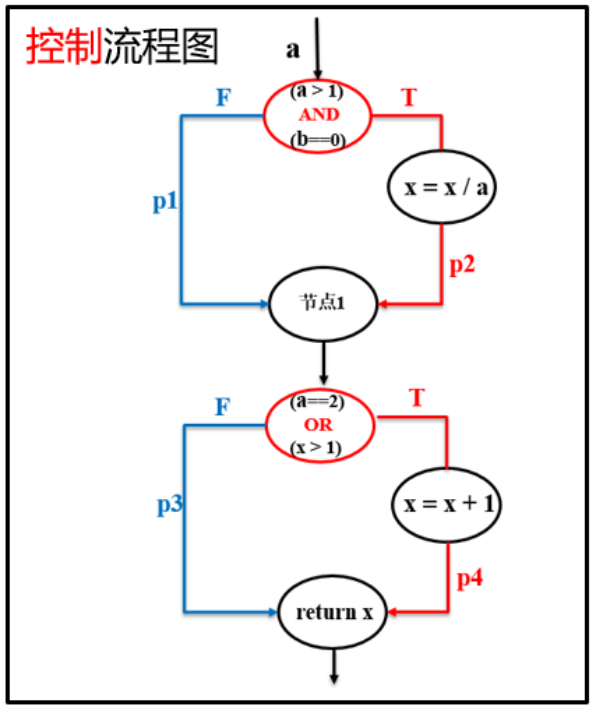
白盒测试：互动小问题

（三）判定覆盖

判定覆盖是否考虑了每个条件的结果呢？

设计测试用例，实现判定覆盖：

测试用例ID	输入			预期输出	覆盖路径
	a	b	x	x	
TC1	1	1	2	3	L14
TC2	3	0	3	1	L23



白盒测试：一个例子

（三）条件覆盖

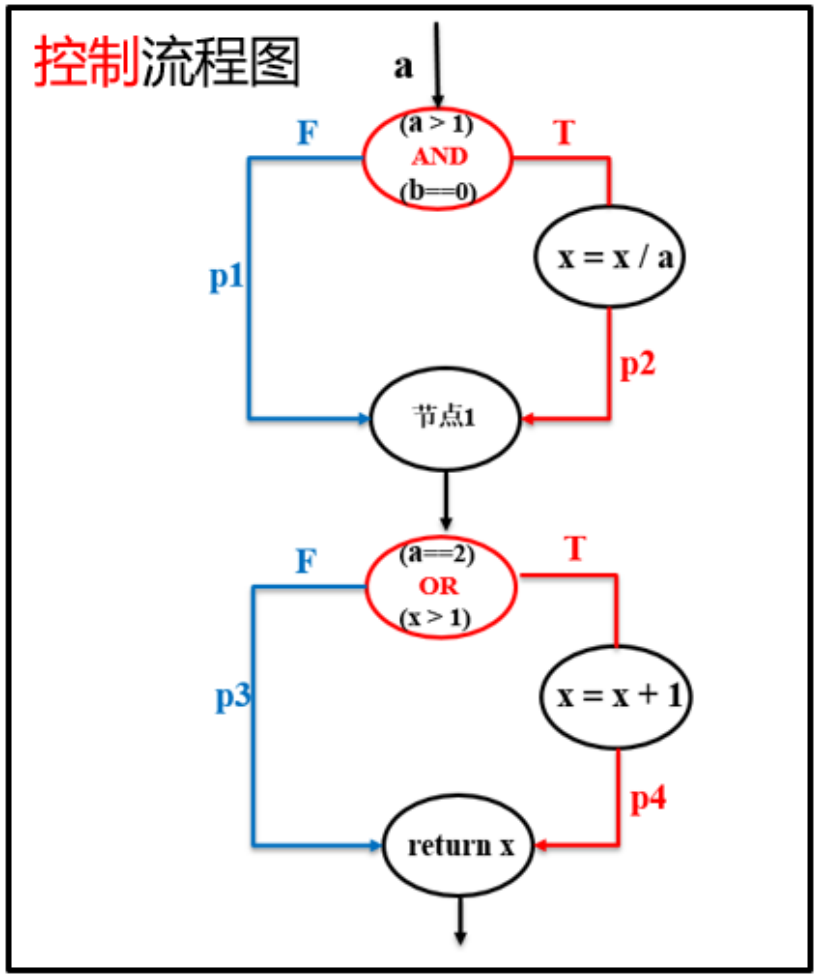
不仅**每个语句**至少执行一次，而且使判定表达式中的**每个条件**都取到各种可能的结果（真、假）。

测试用例ID	输入			预期输出	覆盖路径	覆盖条件
	a	b	x	x		
TC1	2	1	0	1	L14	T1(T) T2(F) T3(T) T4(F)
TC2	1	0	2	3	L14	T1(F) T2(T) T3(F) T4(T)

测试漏洞： x = x / b ✗

- **4个基本逻辑判定条件**

T1	a > 1	T3	a==2
T2	b==0	T4	x > 1



白盒测试：互动小问题

（三）条件覆盖

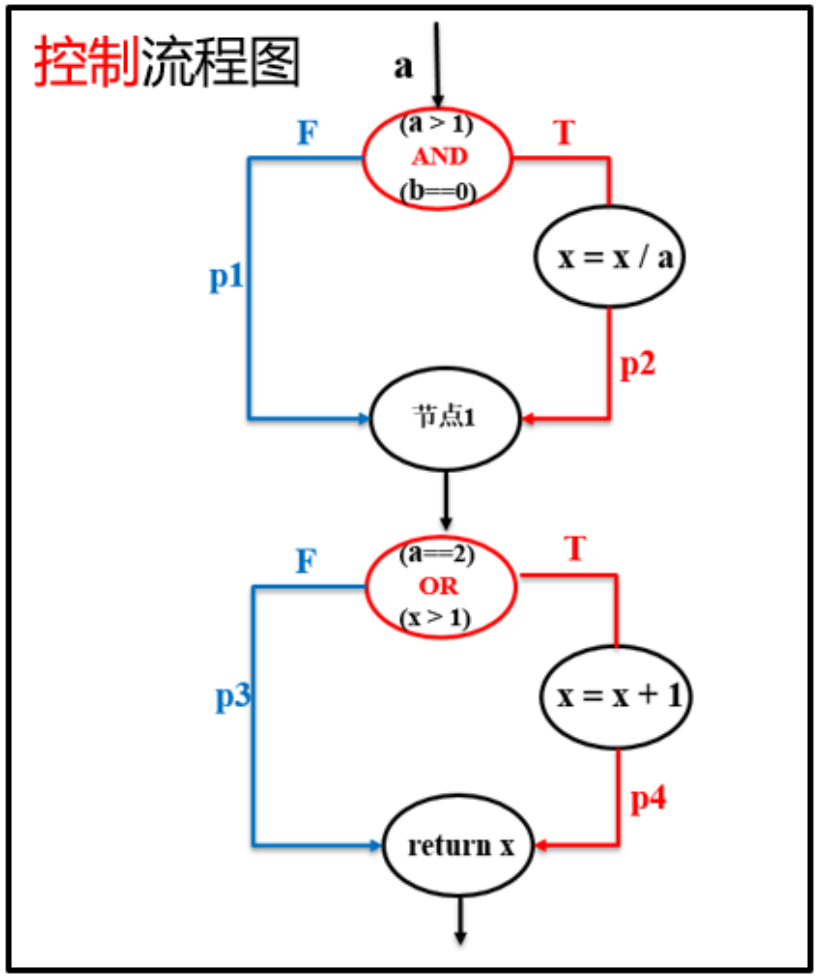
满足条件覆盖是否满足判定覆盖呢？

白盒测试：一个例子

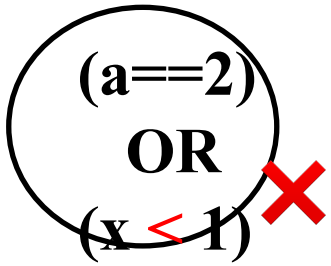
（四）判定/条件覆盖

同时满足判定覆盖和条件覆盖。

测试用例ID	输入			预期输出	覆盖路径	覆盖条件
	a	b	x	x		
TC1	2	0	4	3	L24	T1(T) T2(T) T3(T) T4(T)
TC2	1	1	1	1	L13	T1(F) T2(F) T3(F) T4(F)



测试漏洞：



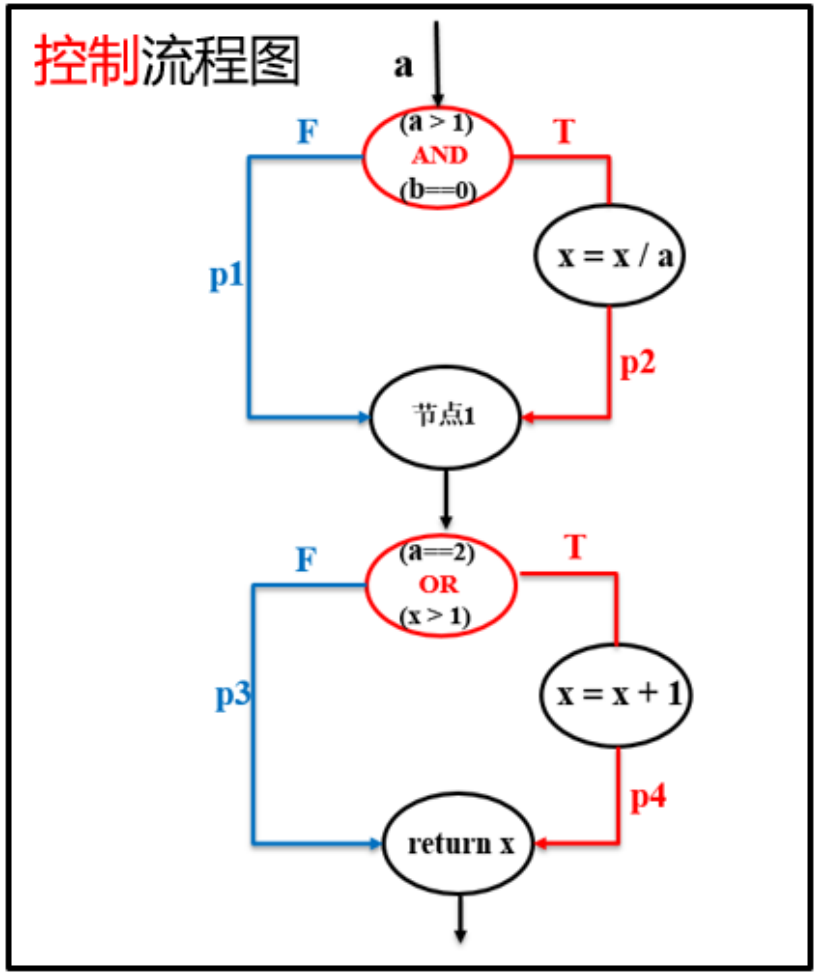
- **4个基本逻辑判定条件**

T1	a > 1	T3	a==2
T2	b==0	T4	X>1

白盒测试：一个例子

（四）判定/条件覆盖

似乎很完美了，但是计算机是怎么对多个条件做出判定的？



测试用例ID	输入			预期输出	覆盖路径	覆盖条件
	a	b	x	x		
TC1	2	0	4	3	L24	T1(T) T2(T) T3(T) T4(T)
TC2	1	1	1	1	L13	T1(F) T2(F) T3(F) T4(F)

白盒测试：一个例子

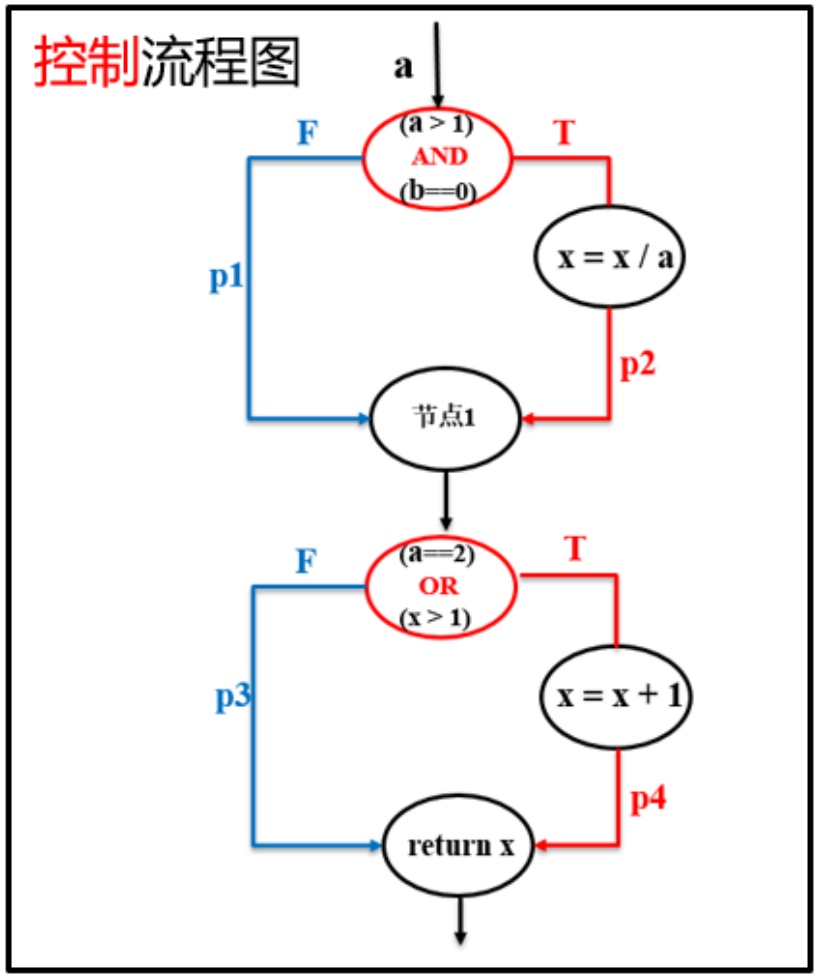
（五）条件组合覆盖

保证程序每个判定节点中，所有简单判定条件的所有可能的**取值组合情况**至少执行一次。

总共8种可能的组合情况：

简单判定条件	
T1:a > 1	T2:b==0
T	T
T	F
F	T
F	F

简单判定条件	
T3:a==2	T4:x > 1
T	T
T	F
F	T
F	F



白盒测试：一个例子

（五）条件组合覆盖

设计用例，实现条件组合覆盖：

可以选择 TC1、TC6、TC9、TC12

缺点：

- 可能存在冗余。如红框内的用例，有相同的覆盖路径。
- 组合数量大，花费时间多

测试用例ID	输入			预期输出	判定条件				覆盖路径
	a	b	x	x	T1	T2	T3	T4	
TC1	2	0	4	3	T	T	T	T	L24
TC2	2	0	2	2	T	T	T	F	L24
TC3	3	0	6	3	T	T	F	T	L24
TC4	3	0	3	1	T	T	F	F	L23
TC5	2	1	2	3	T	F	T	T	L14
TC6	2	1	1	2	T	F	T	F	L14
TC7	3	1	2	3	T	F	F	T	L14
TC8	3	1	2	2	T	F	F	F	L13
					F	T	T	T	不存在
					F	T	T	F	不存在
TC9	1	0	2	3	F	T	F	T	L14
TC10	1	0	1	1	F	T	F	F	L13
					F	F	T	T	不存在
					F	F	T	F	不存在
TC11	1	1	2	3	F	F	F	T	L14
TC12	1	1	1	1	F	F	F	F	L13

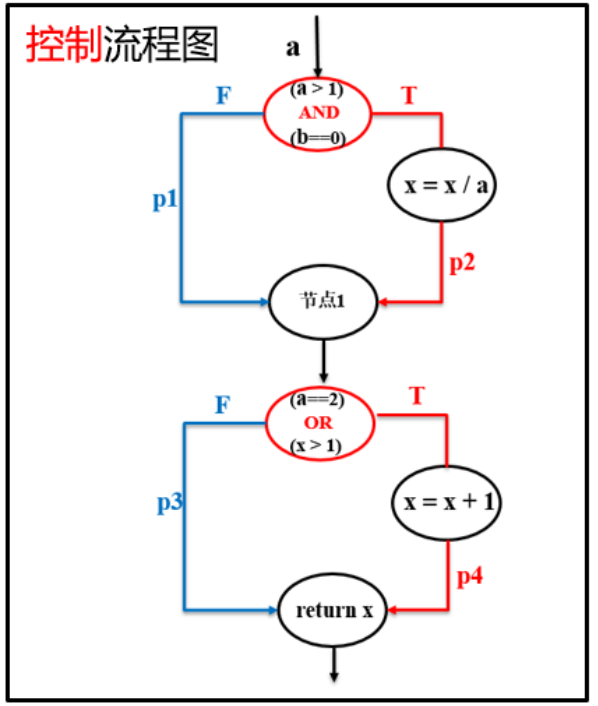
白盒测试：一个例子

（六）路径覆盖

使程序中每一条可能的路径至少执行一次。该策略是最强的， 但一般是不可实现的。

设计测试用例，实现路径覆盖：

测试用例ID	输入			预期输出	覆盖路径
	a	b	x	x	
TC1	2	0	4	3	L24
TC2	1	1	1	1	L13
TC3	1	1	2	3	L14
TC4	3	0	3	1	L23

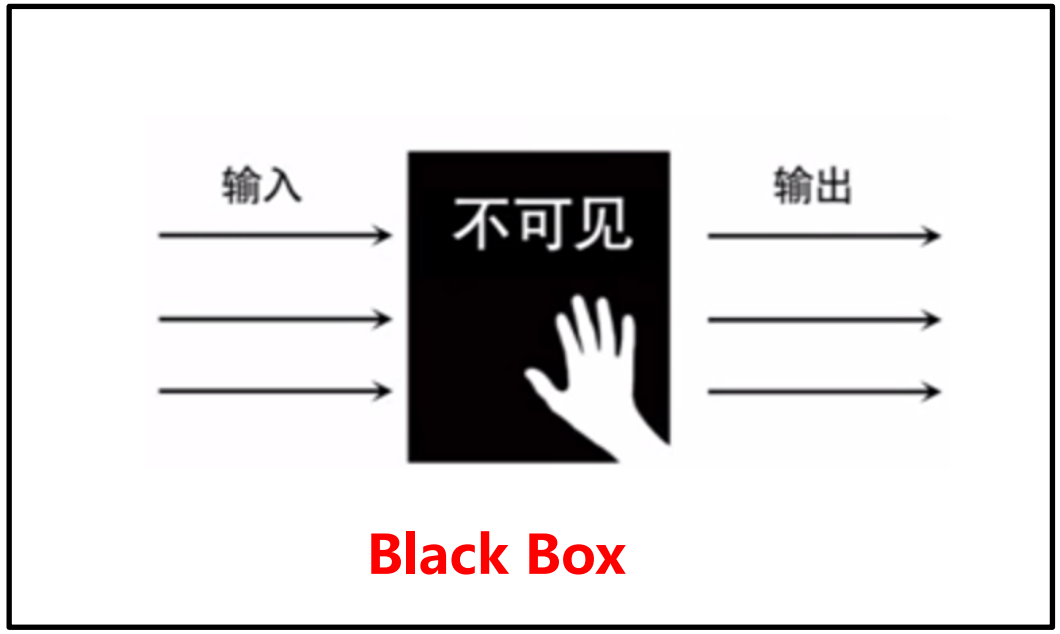


白盒测试：逻辑覆盖方法

覆盖指标	
语句覆盖	至少执行程序中所有语句一次。语句覆盖是最弱的逻辑覆盖准则。
判定覆盖	也称为分支覆盖，至少执行程序中每个分支一次，保证程序中每个判定节点取得每种可能的结果至少一次。
条件覆盖	保证程序中每个复合判定表达式中，每个简单判定条件的取真和取假情况至少执行一次。
判定条件覆盖	保证程序中每个复合判定表达式中，每个简单判定条件的取真和取假情况至少执行一次（条件覆盖），同时保证程序中每个判定节点取得每种可能的结果至少一次（判定覆盖）。
条件组合覆盖	保证程序每个判定节点中，所有简单判定条件的所有可能的取值组合情况至少执行一次。
路径覆盖	使程序中每一条可能的路径至少执行一次。该策略是最强的， 但一般是不可实现的。

黑盒测试：定义

也称作功能测试或数据驱动测试，它着眼于程序外部结构，在完全不考虑程序的内部逻辑结构和内部特性的情况下，测试者在程序接口进行测试，它只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当的接收输入数据而产生正确的输出信息。



优势：

- 方法简单有效
- 可以整体测试系统行为
- 开发与测试可以并行
- 对测试人员技术要求相对较低

劣势：

- 入门门槛低

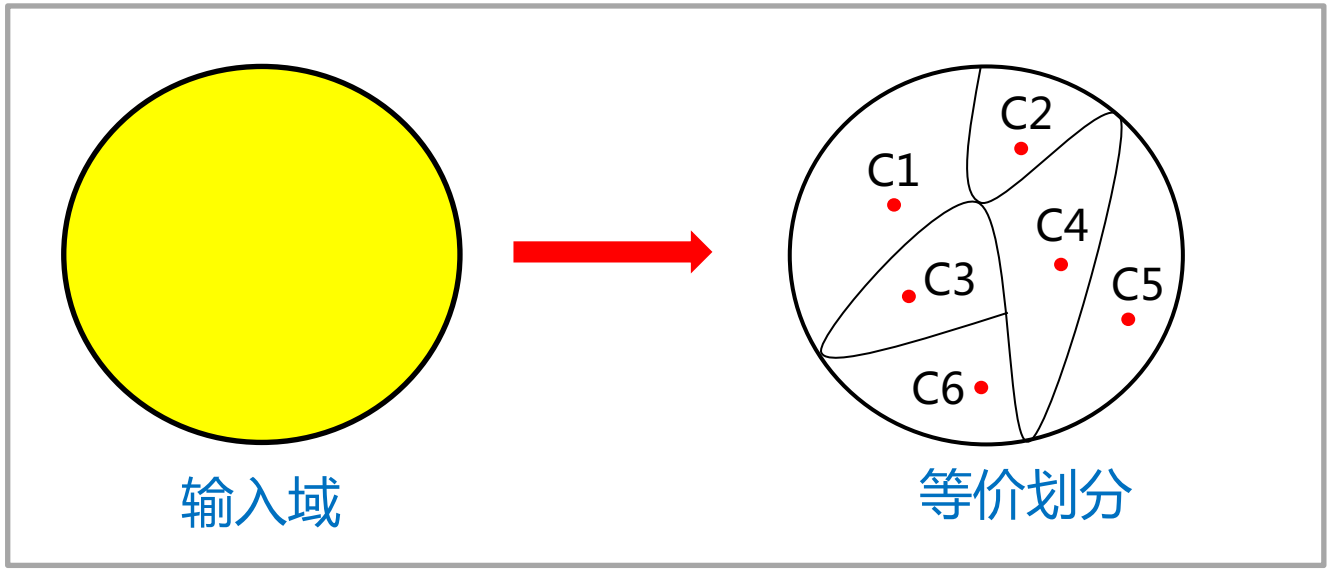
黑盒测试：等价类划分

等价类

输入域的一个子集，在该子集中，各个输入数据对揭示程序中错误是等效的。

等价类测试

将无穷多的数据缩减到有限个等价区域中，通过测试等价区域完成穷尽测试。



- 分而不交
- 合而不变
- 类内等价

黑盒测试：等价类划分

有效等价类

- 指对于软件的规格说明书而言，是合理的，有意义的输入数据所构成的集合。
- 用于检验被测系统是否能够完成指定功能。

无效等价类

- 指对于软件的规格说明书而言，是不合理的，没有意义的输入数据所构成的集合。
- 用于考察被测系统的容错性。

黑盒测试：等价类划分

划分原则：

①如果输入条件规定了一个取值范围或取值个数，则可确定一个有效等价类和两个无效等价类。（如：数量从1到999）

②如果输入条件规定了一个输入值的集合（假定n个），而且软件要对每个输入值进行不同处理，则可确定n个有效等价类和一个无效等价类。

③如果存在输入条件规定了“必须是”的情况，则可确定一个有效等价类和一个无效等价类。（如：标识符第一个字符必须是字母）

④如果输入的是布尔表达式，则可确定一个有效等价类和一个无效等价类。

黑盒测试：一个例子

系统某查询条件是 1990 年 1 月到 2019 年 12 月，由 6 位数字表示，前 4 位表示年份，后 2 位表示月份。

- **确定有效等价类和无效等价类：**

输入条件	有效等价类		无效等价类	
日期的类型和长度	C1	6位数字字符	C2	有非数字字符
			C3	少于6位数字字符
			C4	多于6位数字字符
年份范围	C5	1990~2019	C6	小于1990
			C7	大于2019
月份范围	C8	01~12	C9	等于00
			C10	大于12

黑盒测试：一个例子

- **编写测试用例覆盖所有等价类**
  - 覆盖有效等价类

测试数据	期望结果	覆盖的有效等价类
201408	**输入有效**	**C1、C5、C8**

  - 为什么一次性覆盖**所有**有效等价类？

黑盒测试：一个例子

- **编写测试用例覆盖所有等价类**
  - 覆盖无效等价类

测试数据	期望结果	覆盖的无效等价类
aaa123	无效输入	**C2**
2011	无效输入	**C3**
20140408	无效输入	**C4**
187905	无效输入	**C6**
209811	无效输入	**C7**
200100	无效输入	**C9**
200156	无效输入	**C10**

  - 为什么一次性覆盖**一个**有效等价类？

黑盒测试：边界值分析

是一种最常用的黑盒测试技术。它倾向于选择系统边界或边界附近的数据来设计测试用例，考虑了边界条件的测试用例具有更高的测试回报率。

基本原理

经过长期的测试工作经验表明，大量的缺陷经常发生在输入域或输出域的边界上。因此设计一些测试用例，使程序运行在边界情况附近，这样揭露错误的可能性比较大。

黑盒测试：边界值分析

边界点：

指等价类中那些恰好处于边界的点，可能导致被测系统内部处理机制发生变化的点。

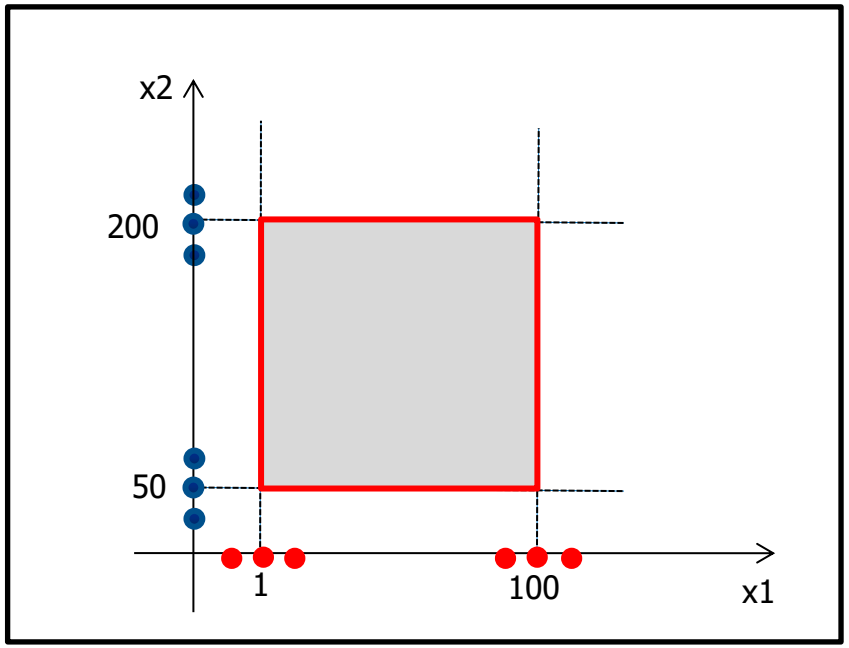
- 举个例子

int Add （ int x1, int x2 ）

1 <= x1<=100

50<= x2<=200

- 需求说明：
  - 对于有效输入，函数返回x1与x2的和；
  - 对于无效输入，函数返回-1。



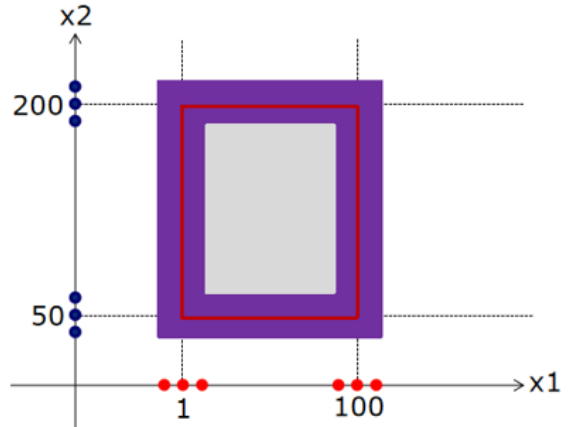
找到每个输入条件的边界点，即可找到系统边界。

黑盒测试：边界值分析

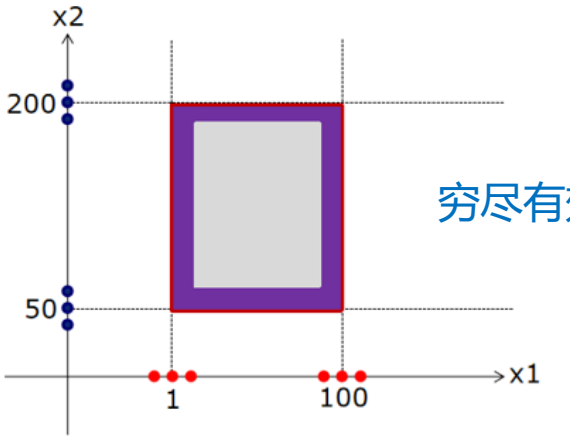
组织测试数据设计测试用例：

用例评价标准：数量、覆盖度、冗余度、缺陷定位能力、复杂度。

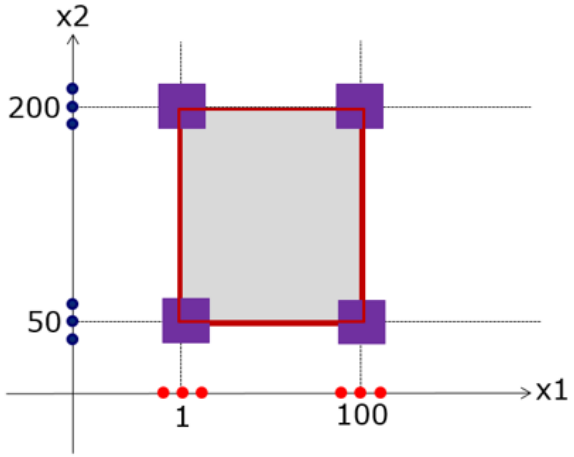
穷尽测试



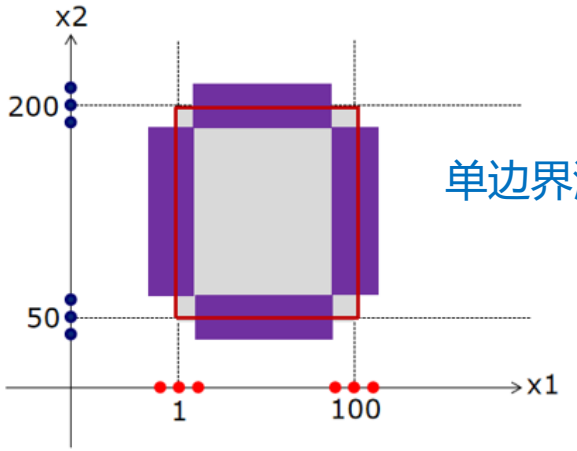
穷尽有效域测试



多边界测试



单边界测试



黑盒测试：边界值分析

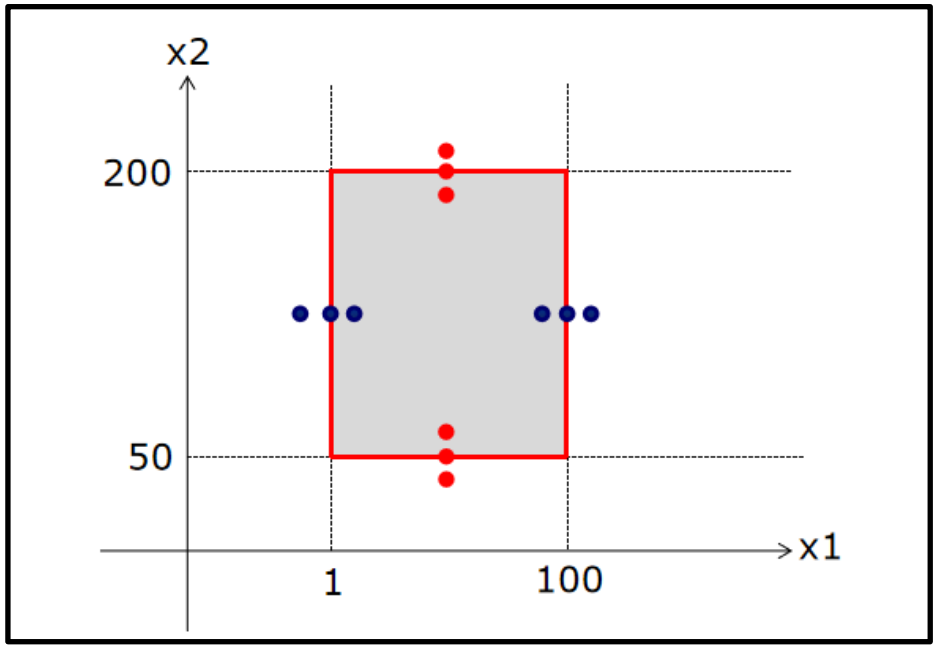
组织测试数据设计测试用例：

- **最终方案：优化后的单边界**

int Add （ int x1, int x2 ）

1 <= x1<=100

50<= x2<=200



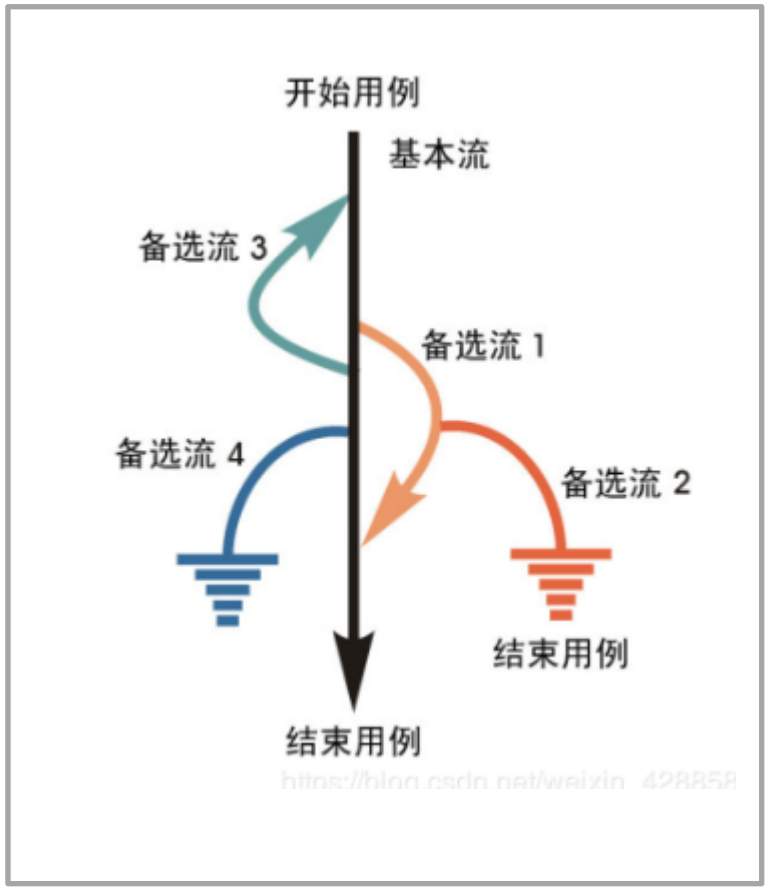
- 用例数量：12
- 覆盖度：100%
- 冗余度：无
- 缺陷定位能力：容易
- 复杂度：低

黑盒测试：场景法

以事件流为核心，主要目的是测试软件的主要业务流程、主要功能的正确性和异常处理能力 。

测试步骤：

- 定义基本流和备选流；
- 定义场景；
- 从场景设计测试用例；
- 输入测试数据，完善测试用例。



黑盒测试：一个例子

举个例子：ATM取款

基本流（正确取款）	
1	插入银行卡
2	验证银行卡
3	输入密码
4	验证密码
5	进入ATM机主界面
6	取款并选择金额
7	ATM机验证
8	更新账户余额并出钞
9	返回主界面

基本流

备选流（出错环节）	
1	银行卡错误
2	密码错误
3	密码3次错误
4	卡内余额不足
5	超出当日可取
6	ATM机余额不足

备选流

黑盒测试：一个例子

➢ **定义场景并设计测试用例**

用例编号	场景	银行卡号	密码	账户余额	当日可取	ATM余额	预期结果
1	场景1：成功取款	V	V	V	V	V	成功取款
2	场景2：银行卡无效	I	N/A	N/A	N/A	N/A	提示错误、退卡
3	场景3：密码错误	V	I	N/A	N/A	N/A	提示错误
4	场景4：密码3次错误	V	I	N/A	N/A	N/A	提示错误、吞卡
5	场景5：账户余额不足	V	V	I	N/A	N/A	提示错误、退卡
6	场景6：总取款金额超出当日可取限额	V	V	V	I	N/A	提示错误、退卡
7	场景7：ATM机余额不足	V	V	V	V	I	提示错误、退卡

注：V 表示有效的数值；I 表示无效的数值；N/A表示不适用。

黑盒测试：一个例子

➢ 输入测试数据，完善测试用例

用例编号	对应场景	前提条件	操作步骤	预期结果
C1	**场景1**	①ATM机余额5000元 ②银行卡号：1234567812345678 ③密码：123456 ④卡内余额：2000元	①插入银行卡 ②输入正确密码 ③进入主页后选择取款1000元	①ATM机输出1000元，提示用户取走并返回主界面 ②ATM余额4000元 ③用户卡内余额1000元
C2	**场景2**	ATM机余额4000元	插入无效银行卡	提示银行卡无效并退卡
C3	**场景3**	①ATM机余额4000元 ②银行卡号：1234567812345678 ③密码：123456 ④卡内余额：1000元	①插入银行卡 ②输入错误密码：654321	提示密码错误，并清空密码
C4	**场景3**	①ATM机余额4000元 ②银行卡号：1234567812345678 ③密码：123456 ④卡内余额：1000元	在C3基础上再次输入错误密码：987654	提示密码错误，并清空密码
C5	**场景4**	①ATM机余额4000元 ②银行卡号：1234567812345678 ③密码：123456 ④卡内余额：1000元	在C4基础上再次输入错误密码：876543	提示密码错误，并没收该卡

单元测试（Unit Testing）

是**代码层面**的测试，是对软件中的最小可测试单元进行检查和验证。单元测试，顾名思义是测试一个“单元”，有别于集成测试，这个“单元”一般是类或函数，而不是模块或者系统。

单元测试可看作是编码工作的一部分，应该由程序员完成，经过了单元测试的代码才是已完成的代码，提交产品代码时也要同时提交测试代码。

单元测试和JUnit

JUnit：一个Java语言的单元测试框架

- 由Kent Beck（极限编程）和 Erich Gamma（设计模式）建立的。
- 是xUnit家族中最成功的的一个。
- 大部分的Java IDE都集成了JUnit作为单元测试工具。
- 官网：http://www.junit.org





工欲善其事必先利其器！

性能测试（Performance testing）

通常收集所有和测试有关的所有性能，被不同人在不同场合下进行使用。 性能测试是一种“正常”测试，主要测试使用时系统是否满足要求，同时可能为了保留系统的扩展空间而进行的一些稍稍超过“正常”范围的测试。

性能测试工具:Load impact



压力测试（Stress Testing）

压力测试是性能测试的一种，它的目标是测试在一定的负载下系统长时间运行的稳定性，但是这个负载不一定是应用系统本身造成的。压力测试尤其关注大业务量情况下长时间运行系统性能的变化（例如是否反应变慢、是否会内存泄漏导致系统逐渐崩溃、是否能恢复）。

压力测试工具:JMeter
官网：https://jmeter.apache.org



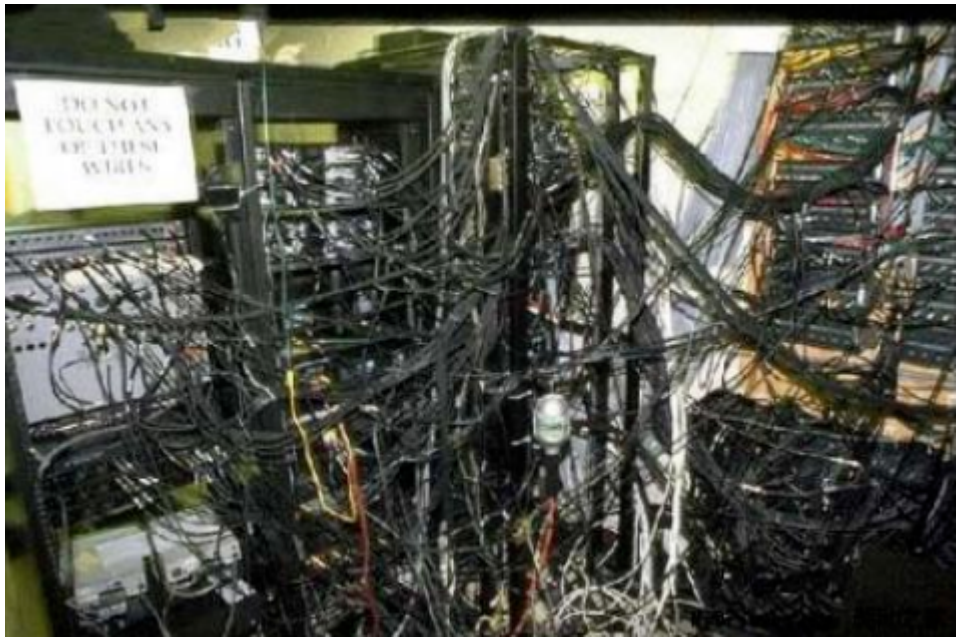
代码覆盖率测试

- **代码覆盖率 = 代码的覆盖程度，一种度量方式**
  - 语句覆盖(StatementCoverage)
  - 判定覆盖(DecisionCoverage)
  - 条件覆盖(ConditionCoverage)
  - 路径覆盖(PathCoverage)

代码覆盖率测试工具:JaCoCo 官网：https://www.jacoco.org/jacoco/trunk/index.html

代码质量

软件质量，不但依赖于架构及项目管理，而且与代码质量紧密相关。代码质量是软件开发不可或缺的一部分。



Bad Code



Good Code

如何评价代码质量

1. **可维护性（Maintainability）**
   - 在不破坏原有代码设计、不引入新的 bug 的情况下，是否能够
   - 快速地修改或者添加代码。
2. **可读性（Readability）**
   - 代码是否符合编码规范、命名是否达意、注释是否详尽、函数
   - 是否长短合适、模块划分是否清晰、是否符合高内聚低耦合等。
3. **可扩展性（Extensibility）**
   - 在不修改或少量修改原有代码的情况下，是否可以通过扩展的

方式添加新的功能代码。

如何评价代码质量

4. **可复用性（Reusability）**
   - 代码中排名第一的坏味道就是代码重复。

     ——Kent Benk 和Martin Fowler
   - 应该编写可重用的、通用的代码，复用已有的代码。
5. **可测试性（Testability）**
   - 是否易于针对代码编写单元测试；
   - 代码可测试性差、单元测试覆盖率低通常意味着代码设计得不够合理。
6. **简洁性（Simplicity）**
   - KISS 原则：“Keep It Simple，Stupid”；
   - 尽量保持代码简单、逻辑清晰。

如何提高代码质量

1. 严格遵循编码规范

阿里巴巴Java开发手册插件

2. 编写高质量的单元测试

3. 代码审查 (Code Review)

4. 开发未动文档先行

5. 持续重构，重构，重构

本讲小结

- 1. 软件测试的定义和分类
- 2. 测试用例的定义和其设计方法
- 3. 白盒测试（定义、控制流程图、逻辑覆盖方法、测试用例设计）
- 4. 黑盒测试（定义、等价类划分、边界值分析、场景法）
- 5. 压力测试，性能测试和代码覆盖率测试
- 6. 代码质量保证